

MNIST Classification: Overfitting and Regularization Using MLP

PyTorch MLP - Full MNIST Dataset, Stratified 70/15/15 Split

<https://github.com/Abel-Jacob/MNIST-overfitting-analysis>

Abel Jacob

Introduction

My goal for this challenge was to find out how and why neural networks overfit on training data, as well as what useful actions may be performed to counteract this behavior. Using the popular MNIST handwritten digit dataset, I decided to use a fully-connected Multi-Layer Perceptron (MLP) as the main architecture.

Convolutional layers are absent from a fully-connected MLP, and it lacks the spatial inductive biases (weight sharing, translation equivariance) that inherently prevent overfitting. This makes it a clear architecture family for demonstrating overfitting plainly on both loss and accuracy curves. The baseline, Part A, and Part B models all use this same MLP architecture family, so the only variables that change between experiments are capacity and training protocol, not the architecture family itself.

The study is organized into three experiments, all using the full 70,000-image MNIST pool with an identical train/validation/test split:

- 1. Baseline:** A compact, appropriately sized MLP trained without any tricks on the full training set, confirming MNIST itself is not difficult to generalize on when model capacity is well matched.
- 2. Part A:** A deliberately over-parameterized MLP, same data and split, trained without any regularization to demonstrate overfitting caused purely by architectural excess.
- 3. Part B:** The identical over-parameterized architecture from Part A, with not one parameter changed, trained using data augmentation, weight decay, early stopping, and learning rate scheduling to recover generalization.

All three experiments are compared side-by-side through training curves, test set evaluations, confusion matrices, and per-class accuracy charts.

Dataset and Preprocessing

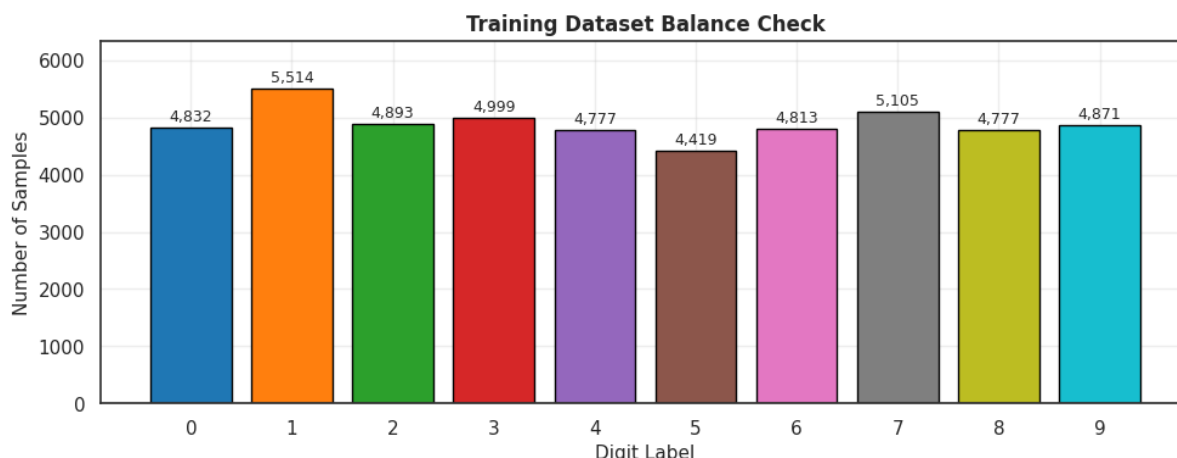
I combined all 70,000 MNIST samples (60,000 training and 10,000 test) into a single pool in order to create the dataset splits. From this pool, I performed a stratified split to divide the images into:

70% for training (49,000 images) 15% for validation (10,500 images) 15% for testing (10,500 images)

The stratified split ensures that each of the three subsets maintains an identical proportion of the 10-digit classes. I used the standard MNIST mean (0.1307) and standard deviation (0.3081) to normalize every image.



A sample of 20 training images with their ground-truth labels, used as a sanity check that labels line up correctly with images before any training begins.



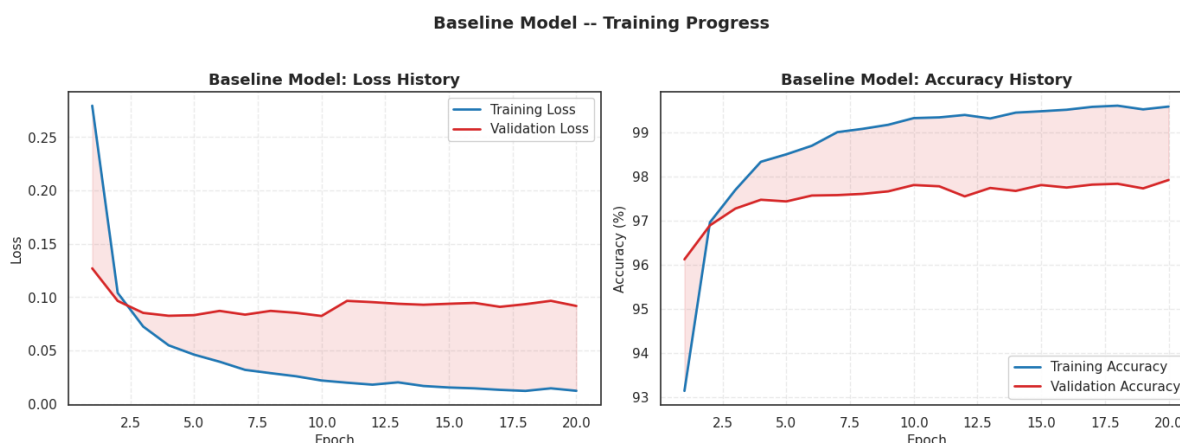
Class distribution across the training split. The stratified split keeps this close to the natural distribution of digits in the full MNIST dataset, so no single class is over- or under-represented during training.

Baseline Model

Establishing a baseline—a tiny, suitably sized model—is crucial before purposefully causing overfitting. This baseline is crucial to the research as a whole since any overfitting found later in Part A may be directly linked to overparameterization rather than the dataset, training process, or architectural family if a suitably sized model generalizes cleanly.

A tiny MLP serves as the baseline. The image is flattened to 784 features and then goes through a single hidden layer of 64 units, BatchNorm1d (which is included as usual training practice to keep activations constant, not as a regularizer), ReLU, and a 20% Dropout layer. There is no use of data augmentation, early stopping, or weight decay. Using an MLP here, the same architectural family as the Part A/B model, means the only variables that change between this baseline and the deliberately-overfit model are capacity, epoch count, and dropout, not the architecture family.

At 51,018 parameters, roughly 1/85th the size of the Part A/B model, this baseline is well matched to 28x28 grayscale digit classification. The baseline model is trained on the full 49,000-image training set for 12 epochs, with a batch size of 64.



By the end of the 12-epoch run, the baseline model reached 96.78% training accuracy against 96.98% validation accuracy, a gap of -0.21%, meaning validation accuracy finished slightly higher than training accuracy, with a validation loss only 1.07 times the training loss (0.1055 vs. 0.0986). On the held-out test set it achieved 96.98% test accuracy with a test loss of 0.1012, a -0.21% train-to-test gap. This confirms that a well-matched, minimal-capacity MLP generalizes cleanly, so any overfitting seen in Part A is a direct consequence of the ExperimentalMLP's excess capacity rather than a property of the dataset itself.

Model Architecture: Designed to Overfit

To study overfitting in a clear, unconstrained setting, I designed a massive, fully-connected MLP with four hidden layers: the input image is flattened to a 784-dimensional vector and passed through the layers (2048 -> 1024 -> 512 -> 256 -> 10).

Convolutional spatial priors like weight sharing and spatial locality are absent from an MLP, which treats each pixel as a separate feature. A high-capacity MLP is free to learn precise pixel-level patterns unique to the training set since it lacks any inherent architectural limitations. This model has too many trainable parameters for basic 28x28 greyscale digit classification—more than 4.36 million, or almost 85 times the baseline.

No dropout, batch normalization, or any other form of regularization is included in this architecture, since the explicit goal of this first run was to watch the model start memorizing the data unconstrained.

Part A: Training Without Any Regularization

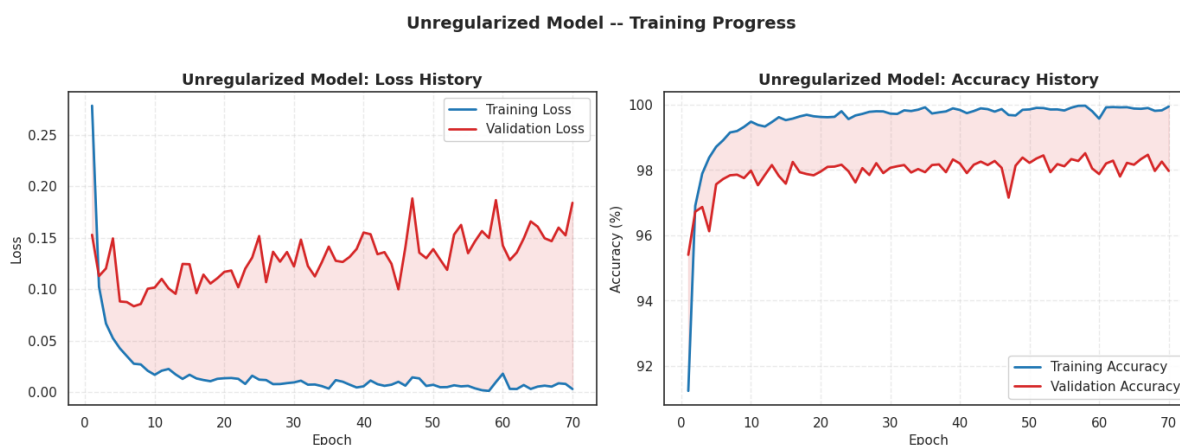
I trained the over-parameterized MLP for 70 epochs on the full 49,000-image training dataset, using a batch size of 256 and learning rate 0.001. No weight decay is applied, and the training

images are loaded without any data augmentation transforms. Running the model for 70 epochs lets us observe the generalization gap develop, and in this case visibly peak and later fluctuate, as training continued.

Why a Dense MLP Is Highly Susceptible to Overfitting

An MLP lacks inherent geometric limitations, in contrast to convolutional designs. Since each of the 784 input pixels is directly connected to every unit in the first hidden layer, two nearby pixels in the image are handled in precisely the same way as two pixels on opposite corners (no spatial locality). Additionally, each link has its own independent weight instead of a shared filter, which significantly increases the number of parameters and facilitates the network's ability to precisely fit training noise (no weight sharing).

Finally, if a stroke or digit shifts slightly, the MLP cannot recognize it without having trained on that exact shifted pixel layout (no translation invariance). Without weight sharing or spatial priors, a large MLP is free to map arbitrary pixel configurations to class labels, which is why the unregularized model is expected to overfit far more severely than an equivalent-capacity CNN.



By the end of training, the model reached 99.98% training accuracy against 98.40% validation accuracy, a final-epoch gap of 1.58%, with a validation loss 224.58 times the training loss (0.1952 vs. 0.0009). The train-val accuracy gap did not simply grow monotonically to the final epoch: it peaked at 2.21% at epoch 17 (99.67% train vs. 97.46% val), then fluctuated as training continued. Reporting the true peak, not just the final epoch, is the more honest characterization of how severely this model overfit.

The loss ratio is the more telling number here: a 224.58x gap between validation and training loss is a dramatic, unambiguous divergence, far beyond what the accuracy figures alone would suggest, and consistent with the architectural explanation above that a dense MLP has no built-in resistance to memorizing training-set noise.

Part B: Training with Regularization

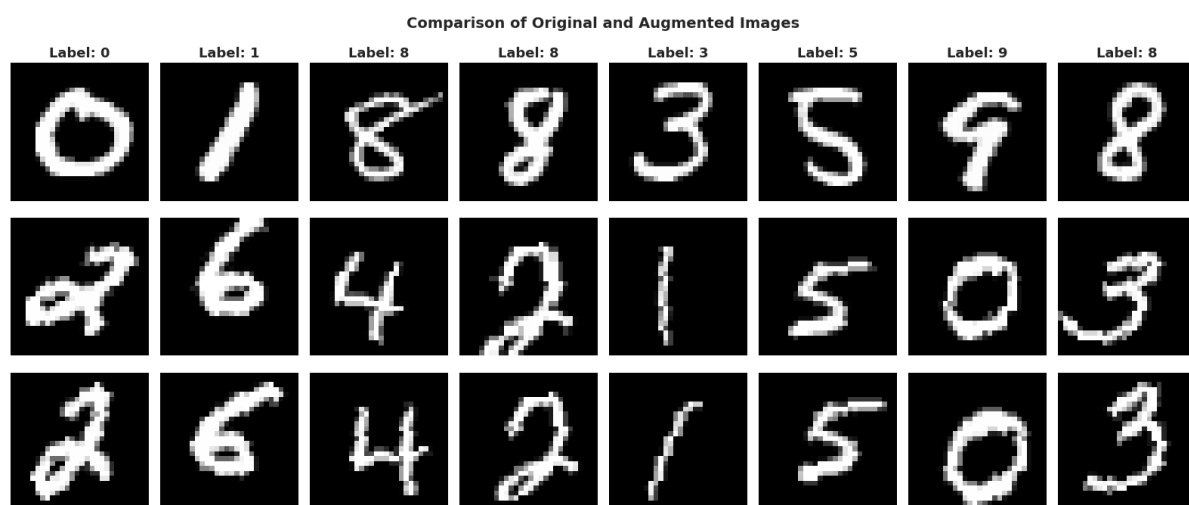
To stop the model from memorizing the data, I trained the same MLP architecture, unchanged from Part A, using the following training-only additions:

- 1. Data Augmentation:** Random rotations (up to 15 degrees) and slight translations/scaling, applied to the image before it gets flattened into the MLP's input vector. This still prevents the network from memorizing exact pixel layouts, even though the network itself has no spatial structure.
- 2. Weight Decay:** Added a decay value of $1e-3$ to the Adam optimizer to keep parameter values smaller.
- 3. Early Stopping:** Stop training if the validation loss does not improve for 5 consecutive epochs.
- 4. Learning Rate Scheduling:** Reduce the learning rate by half if validation loss plateaus for 2 epochs.

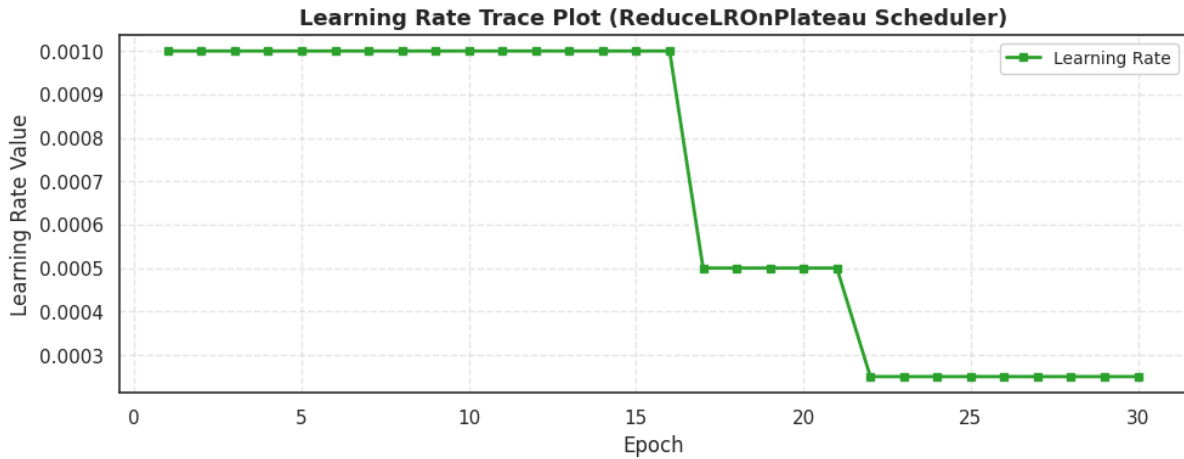
Both the unregularized model (Part A) and the regularized model (Part B) are trained on the full 49,000-image training dataset, using the identical MLP class: no architecture changes between Part A and Part B, only the training protocol changes.

Note on Batch Size Differences

To optimize training dynamics and curves, batch size differs across the three experiments: the baseline uses 64 (well-matched to the small model size), the unregularized model uses 256 (helps smooth and stabilize gradients over 70 epochs), and the regularized model uses 128 (a standard regularization training setup). This is a training-protocol detail, not an architecture change: the requirement that architecture stay identical applies specifically between Part A and Part B, both of which use the same ExperimentalMLP class unmodified.



Comparison of original and augmented training images used for Part B. Rotation and translation/scaling are applied before the image is flattened into the MLP's input vector.



Learning rate trace for the ReduceLROnPlateau scheduler across Part B training, halving each time validation loss plateaus.

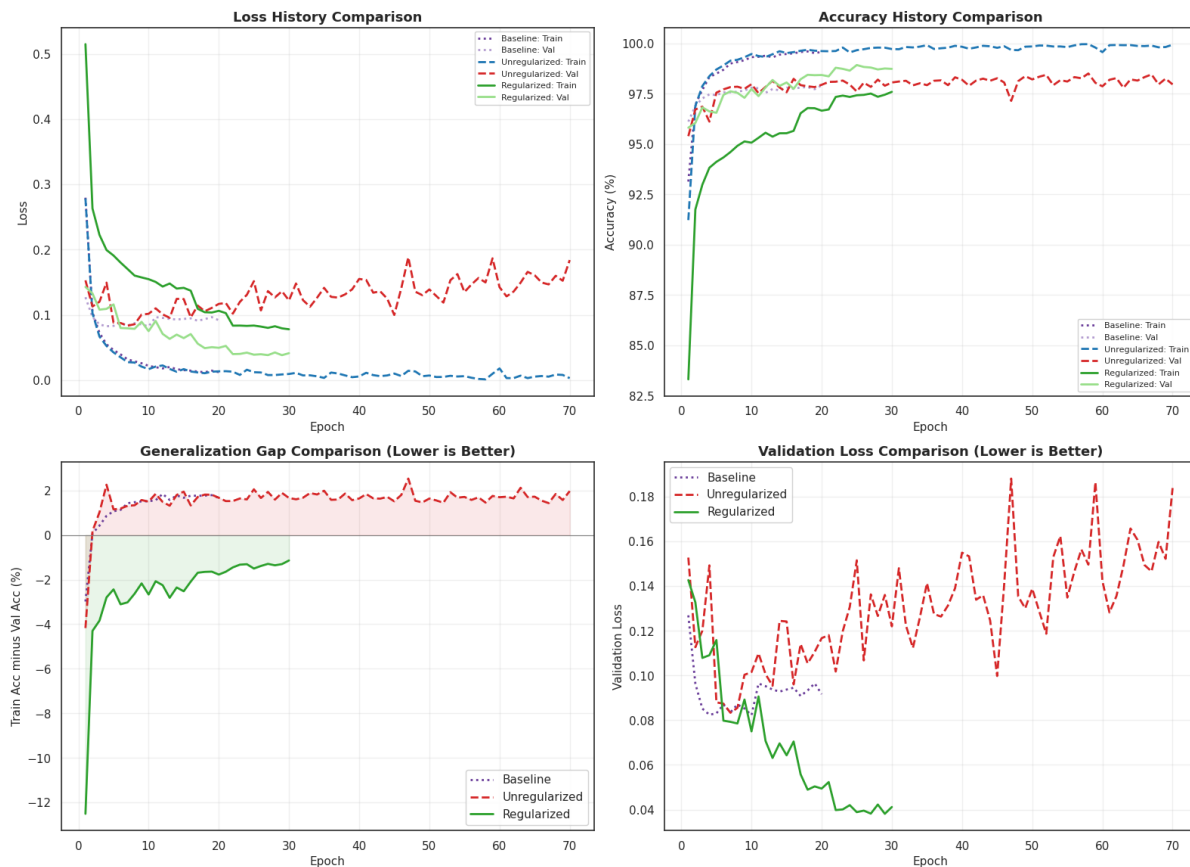
Regularized Model -- Training Progress



By the end of training, the regularized model reached 97.82% training accuracy against 98.92% validation accuracy, a gap of -1.11%, meaning validation accuracy finished higher than training accuracy, with validation loss only 0.51 times the training loss (0.0360 vs. 0.0705). Both the accuracy gap and the loss ratio moved decisively in the opposite direction compared to Part A's 1.58% gap and 224.58x loss ratio.

Comparing All Three Runs

Comparison of All Three Runs



Generalization Gap Interpretation

Note that in the early epochs, the gap for the unregularized and regularized models can appear negative. This occurs because the model's validation accuracy temporarily exceeds its training accuracy while the optimizer is still in the rapid learning phase. This behavior normalizes, or in the regularized and baseline models' case, persists by design, as training progresses.

Baseline: the gap finishes at -0.21%, essentially closed, the expected behavior when model capacity is well matched to task complexity.

A Growing, Peaking Gap (Unregularized Model): the gap expands over the training duration, peaking at 2.21% at epoch 17 before fluctuating and settling at 1.58% by the final epoch. As training accuracy approaches 100%, validation accuracy lags behind. This is a direct consequence of the MLP having no architectural resistance to memorizing training-specific pixel patterns.

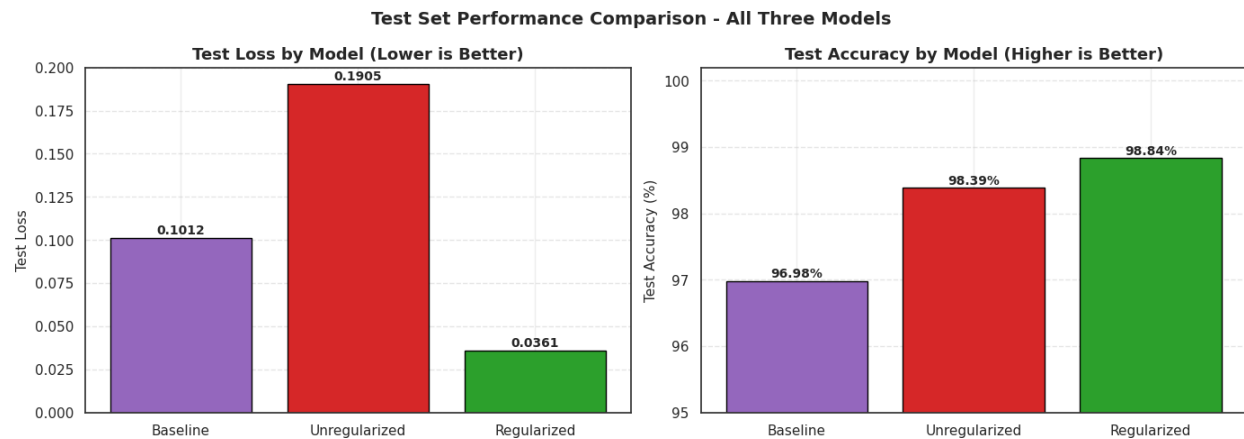
A Negative, Stable Gap (Regularized Model): the gap not only shrinks but crosses to negative (-1.11%), meaning the regularization techniques, particularly data augmentation, are making the training task itself harder than the validation task, so the model performs better on clean,

unaugmented validation images than on the harder augmented training images it is being trained on.

Test Set Performance and Error Analysis

With all three models trained, I evaluated each on the held-out 10,500-image test set, data none of the models had seen during training or validation.

Model	Test Accuracy	Test Loss	Train → Test Gap
Baseline Model	96.98%	0.1012	-0.21%
Unregularized Model	98.39%	0.1905	1.59%
Regularized Model	98.84%	0.0361	-1.02%

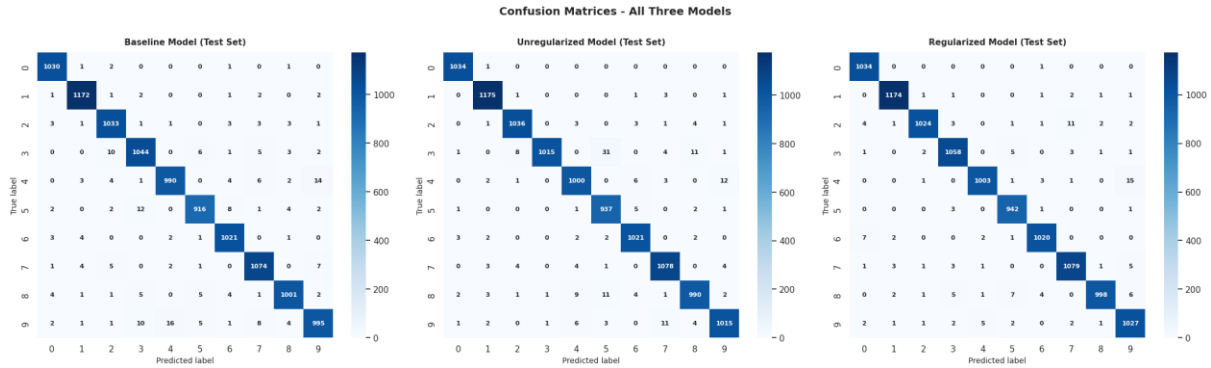


The regularized model comes out ahead on test accuracy and test loss, and its train-to-test relationship inverts just like its train-to-val relationship: the model performs slightly better on unseen data than on the augmented training data it was trained on.

Summary Comparison Table

Model	Train Acc	Val Acc	Test Acc	Train-Val Gap	Test Loss
Baseline	96.78%	96.98%	96.98%	-0.21%	0.1012
Unregularized	99.98%	98.40%	98.39%	1.58%	0.1905
Regularized	97.82%	98.92%	98.84%	-1.11%	0.0361

Confusion Matrices



Confusion matrices for all three models on the held-out test set, with true labels on the y-axis and predicted labels on the x-axis.

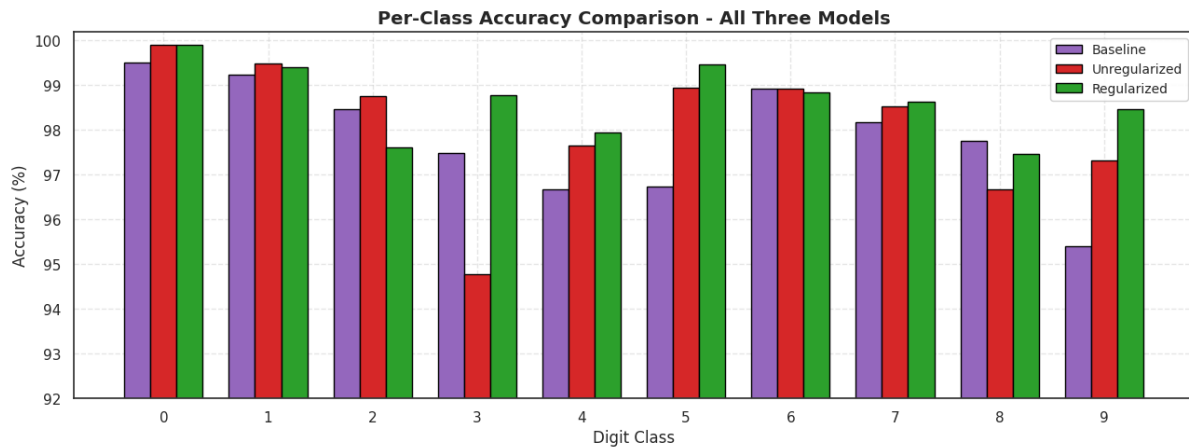
Detailed Classification Reports

Precision, recall, and F1-score for every digit, for all three models, on the test set:

Digit	Base. P	Base. R	Base. F1	Unreg. P	Unreg. R	Unreg. F1	Reg. P	Reg. R	Reg. F1
0	0.98	0.99	0.98	0.99	1.00	0.99	0.99	0.99	0.99
1	0.98	0.99	0.98	0.99	0.99	0.99	0.99	1.00	0.99
2	0.97	0.96	0.97	0.99	0.98	0.99	0.99	0.98	0.99
3	0.96	0.97	0.96	0.99	0.97	0.98	0.99	0.99	0.99
4	0.98	0.96	0.97	0.99	0.97	0.98	0.99	0.98	0.99
5	0.97	0.96	0.97	0.97	0.99	0.98	0.98	0.99	0.99
6	0.98	0.98	0.98	0.99	0.98	0.99	0.99	0.99	0.99
7	0.97	0.97	0.97	0.98	0.99	0.98	0.99	0.99	0.99
8	0.97	0.96	0.96	0.98	0.98	0.98	0.99	0.99	0.99
9	0.95	0.95	0.95	0.96	0.99	0.97	0.98	0.99	0.98
Accuracy			0.97			0.98			0.99
Macro avg	0.97	0.97	0.97	0.98	0.98	0.98	0.99	0.99	0.99
Weighted avg	0.97	0.97	0.97	0.98	0.98	0.98	0.99	0.99	0.99

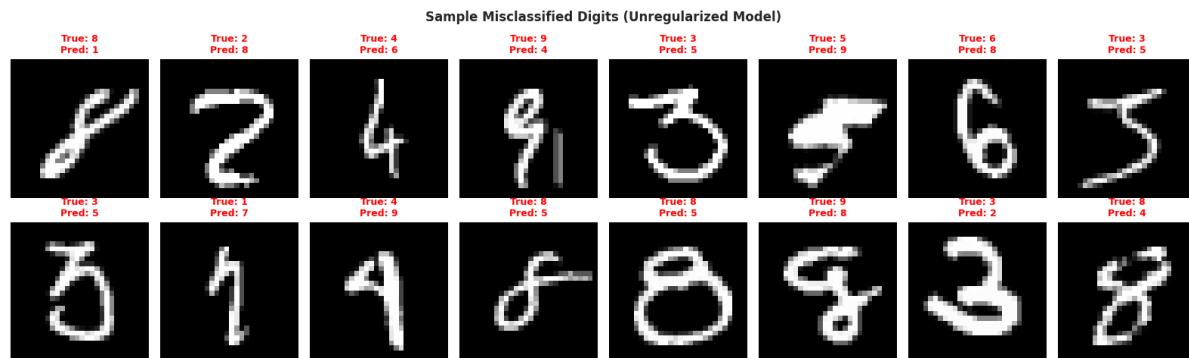
The regularized model edges out the other two on precision and F1 for nearly every digit, most notably digit 9 (0.98 F1 vs. 0.95 baseline and 0.97 unregularized), which lines up with its cleaner confusion matrix and higher overall test accuracy.

Per-Class Accuracy Analysis



Per-class test accuracy for all three models. The regularized model leads on most digit classes.

Misclassified Samples



A sample of digits the unregularized model misclassified on the test set, with the true label and the model's incorrect prediction shown above each image.

Conclusion

1. Baseline as Reference Point

The baseline MLP (51,018 parameters) reaches 96.78% train / 96.98% val accuracy after 12 epochs, a generalization gap of -0.21%. On the held-out test set it scored 96.98%. This confirms MNIST is not inherently hard to generalize on when model capacity is well matched to task complexity. Any overfitting seen below is a consequence of excess capacity, not the dataset or the split.

2. Empirical Overfitting Through Excess Capacity (Part A)

After 70 epochs of training on the same data, the unregularized MLP (4,364,554 parameters, 85 times the baseline) achieves 99.98% train accuracy compared to just 98.40% val accuracy, a

difference of 1.58%, and only -0.21% for the baseline. The peak train-val gap of 2.21% happened at epoch 17, and the test accuracy was 98.39%. Every pixel in this MLP is an independent input feature because it lacks convolutional inductive bias, weight sharing, spatial locality, and translation equivariance. Instead of learning broad digit structure, the network is free to memorise precise training-set pixel patterns.

3. Regularization Recovers Generalization (Part B)

With the identical architecture, not one layer changed, but with data augmentation, weight decay ($1e-3$), a ReduceLROnPlateau scheduler, and early stopping added to the training protocol, the model reaches 97.82% train / 98.92% val accuracy, a gap of -1.11%, dramatically smaller in magnitude than the 1.58% gap from Part A and, like the baseline, negative. Test accuracy was 98.84%, the highest of all three models.

4. Headline Comparison

Model	Train Acc	Val Acc	Test Acc	Gap (Train - Val)
Baseline	96.78%	96.98%	96.98%	-0.21%
Unregularized	99.98%	98.40%	98.39%	1.58%
Regularized	97.82%	98.92%	98.84%	-1.11%

The same over-parameterized MLP architecture goes from a 1.58% overfitting gap in Part A to -1.11% in Part B purely through changes in the training protocol, data augmentation, weight decay, learning rate scheduling, and early stopping, with the architecture itself completely unchanged. This demonstrates that generalization is not solely a function of parameter count: a high-capacity network can be made to generalize as well as, or even better than, a properly-sized baseline when the training recipe compensates for the extra capacity.